Shell Scripting, System Administration & System Programming

Due to 17 October 2026

يعد الغش مخالفة أكاديمية وفقا للوائح والقوانين المعمول بها في جامعة قطر، وقد تصل عقوبة هذه المخالفة في بعض الحالات إلى الفصل النهائي من الجامعة وعلى الطلاب تجنب القيام أو المشاركة في أي عمل يخالف ميثاق النزاهة الأكاديمية وإجراءات الاختبارات المعمول بها في جامعة قطر.

Cheating is an academic violation according to Qatar University rules and regulations, and in some cases, it may result in final dismissal from the University. Students should not under any circumstances commit or participate in any cheating attempt or any act that violates the student code of conduct.

It is the right of the instructor to test the student's understanding of the project in any way during the demo and discussion session. So, a project that is 100% working might be graded (-100%) due to the student not being able to explain a functionality they have implemented.

Project Objectives:

By the end of this project, you will be able to:

1. **Automate user, group and privilege management** on a Linux system using loops and shell scripts.
2. **Install, configure and verify core services** (MySQL, SSH, FTP, Apache), control them with systemd, and capture installation errors in log files.
3. **Search, audit and back up a file system** based on size and permission criteria, and produce compressed, timestamped archives.
4. **Diagnose network configuration** - interfaces, routing, DNS, traceroute and end-to-end reachability - entirely from the command line.
5. **Deploy a centralized logging (syslog) server** and configure a remote client to forward its system events to it, so that all machines are audited from one place.
6. **Access a remote Linux host securely** using SSH, SFTP and HTTP clients, and transfer reports between machines.
7. **Apply system programming concepts in C** - process creation, matching parallelism to the CPU core count, and child-to-parent reporting through log files.
8. Work as a team, evidence your work with screenshots, and defend every line of code you submit.

Environment Setup:

- **Virtual Machines:** Use **Ubuntu 26.04 LTS Desktop** for two VMs:
  - **VM1 – ALPHA (Server):** Hosts the user accounts, the shared services (SSH, MySQL, FTP, Apache) and the centralized log server.
  - **VM2 – BETA (Client):** Used for connectivity testing, remote access, system reporting and log forwarding.

- **Networking:** Both VMs must be on the same network and must be able to reach each other by IP address and by hostname.

First Part: Shell Scripting

ALPHA-VM (Server) Tasks:

Task 1: User and Group Management

- Create 10 users together with their passwords and home directories. Use a loop to do this.
- All users must have root privileges. [Note: they must be added to the sudo group].
- Verify the result by displaying the created accounts and their group memberships.
- **Script Requirement:** Automate all of the above in a Bash script (`Users.sh`). If a user already exists, the script must report it instead of failing.

Task 2: Service Installation and Error Logging

- Update the system, install MySQL, restart the MySQL service, and display a message that the system will be reloaded.
- Install SSH, FTP and the Apache Web Server.
- Configure MySQL, SSH and Apache to start automatically on boot, then check and display the status of each service.
- Any error must be saved in a log file (`Installation.log`).
- **Script Requirement:** Script (`Install.sh`).

Task 3: File Search and Administrator Notification

- Find the files in your account that are larger than 1M.
- Store the results in a file (`bigfile`) along with the search date and the number of files found with that size.
- If (`bigfile`) is not empty, the script must email the system administrator — use your own personal address (for example `yourname@gmail.com` or `yourname@live.com`) — with a message about the contents of the file.
- [Note: sending mail to an external address requires a mail client configured with an SMTP relay (for example `msmtp` with your own mail account). Configure it first, then show the received email in a screenshot].
- **Script Requirement:** Script (`Search.sh`).

Task 4: Network Configuration and Diagnostics

- Display the network configuration information, the routing table and the hostname.
- Test your local DNS server, trace the route to google.com, and finally ping google.
- All information is saved in a log file (`network.log`) and displayed on the screen.
- [Note: if the network tools are not installed in the system, then your shell script needs to install them first].
- **Script Requirement:** Script (`Network.sh`).

Task 5: File System Auditing and Backup

Write one script that contains three subroutines (functions):

- First subroutine:
  - Get the disk usage of the home directory (number of used KB), sort the output in reverse order and save the result in `/tmp/Disk_Usage.txt`.
  - Create `/tmp/cpu_info.txt` containing ONLY the CPU specification of the running system (Architecture, # of CPUs, Model name, Vendor ID).
  - Save the kernel output related to memory and hard disk ONLY into `/tmp/Messages_Log.txt`. [Note: access to kernel messages is restricted on Ubuntu 26.04 — use elevated privileges, or read `/var/log/kern.log` once the logging service of Task 6 is installed].
  - Create `/tmp/Message_count.txt` showing the total word count of the memory lines and of the hard disk lines taken from `/tmp/Messages_Log.txt`.
  - Add the three files to a compressed tar file called `Phase1.tar.bz2` and copy it to a directory whose name is the time of the backup (example: 30055503 → day 30, hour 05, minute 55, second 03).
- Second subroutine – check all the files in your home directory for the following:
  - Files with read and execute permissions for the owner only: make a copy of the file and redirect that copy to a backup directory. [Note: the directory name must be the date and time of the backup, example 260929055503 → year 26, month 09, day 29, hour 05, minute 55, second 03].
  - Change the permission of those files to read permission only for the owner, for the backup directory and its contents.
- Third subroutine:
  - Verify the work of the second subroutine: display the number of files inside the backup directory whose permissions are NOT read-only for the owner, and report each of them as an error (the count must be zero).
  - Display the total number of files that were backed up and the total size of the backup directory.
  - Display a message indicating the end of the execution along with the time.
- **Script Requirement:** Script (`FileSystem.sh`).

Task 6: Centralized Log (Syslog) Server

ALPHA must act as the central log server of the lab: every event produced by BETA is collected and stored on ALPHA, exactly as a syslog server does in a network.

- Install and enable the system logging service (`rsyslog`) and configure it to receive remote messages over UDP and TCP port 514.
- Store the messages received from each client in a separate file named after the sending host, for example `/var/log/remote/<hostname>/<hostname>.log`.
- Allow port 514 through the firewall for the BETA VM only. [Note: allow SSH through the firewall BEFORE enabling it, otherwise you will lock yourself out of ALPHA].
- Restart the service and confirm that it is listening on port 514.
- Generate a local test message using `logger` and show that it is recorded.
- Configure log rotation for `/var/log/remote` so that the files rotate daily and are kept for 7 days.

- **Script Requirement:** Script (`LogServer.sh`) must save its own output to `/var/log/logserver_setup.log` and print the final service status.

BETA-VM (Client) Tasks:

Task 1: Continuous Connectivity Testing

- Continually test connectivity to ALPHA using ping every 5 minutes and append the results, with a timestamp, to `/tmp/SecondVM.txt`.
- **Script Requirement:** Script (`Connectivity.sh`). Explain in the demo how you scheduled the repetition.

Task 2: Remote Access and Web Verification

- SSH to ALPHA. To login, use any one of the 10 users created in Task 1 of ALPHA.
- *[Note: a script cannot type a password. Before writing this script, generate an SSH key pair on BETA and install the public key in the ALPHA user account, so that BOTH the SSH and the SFTP steps run without any password prompt].*
- Browse the `index.html` page hosted on the ALPHA web server from BETA and save the returned page.
- SFTP to ALPHA and upload the system information file produced in Task 3.
- **Script Requirement:** Script (`Remote.sh`).

Task 3: System Information Report

- Collect system information about BETA: disk, CPU, memory, installed applications and network interface card information.
- Write everything into one timestamped report file, ready to be transferred to ALPHA.
- **Script Requirement:** Script (`SysInfo.sh`).

Task 4: Log Forwarding Client

- Set a clear hostname for BETA so that its messages can be identified on the log server.
- Install the logging service (`rsyslog`) on BETA if it is not present, and configure it to forward all of BETA's system messages to ALPHA on port 514.
- Restart the service and generate at least three test events ON BETA: a `sudo` command executed on BETA, a failed SSH login attempt INTO BETA (for example, connect from ALPHA to BETA with a wrong user name or password), and a manual `logger` message.
- *[Note: only the events produced by BETA are forwarded. A failed login from BETA into ALPHA is recorded by ALPHA itself and does not prove that forwarding works].*
- Verify on ALPHA that the three events appear inside `/var/log/remote/`.
- **Script Requirement:** Script (`LogClient.sh`) must first check that ALPHA is reachable on port 514 and exit with a clear error message if it is not.

Second Part: System Programming in C (on ALPHA)

Task 1: Parallel Task Execution with Sibling Processes

- Determine the number of processor cores: the server should ascertain the number of available processor cores. This information is essential for optimizing parallel processing, as it allows the server to tailor its process creation to the capabilities of the hardware.
- Create siblings based on the core count: the server creates an equal number of sibling processes, so that each core handles a separate sibling. [For simplicity: for process creation always consider half of the cores].
- Each sibling is assigned randomly one of the following tasks:
  - Create File
  - Delete File
  - Copy File
  - Display File Information
  - Search for a Word in a File
  - List Directory Contents
- Pass all arguments through the command line (CLI).
- The parent creates a log file; siblings report to the parent by logging their successful task along with the time.
- Once done, the parent prints the contents of the log file on the screen.

Instructions & Deliverables: Please read carefully

1. **Project Report (MS-Word document):**
   - Cover page with the group member details (Student Name, Student ID) and the section number.
   - A task distribution table specifying the tasks assigned to each member and the contribution percentage of each member.
   - The body of the report must contain **SCREENSHOTS ONLY** - at least one clear screenshot for every task and every functionality, showing the command/script that was executed and its output.
   - Every screenshot must carry a caption identifying the VM (**ALPHA** or **BETA**), the script name and the task number, and the terminal prompt with the hostname must be visible in the image.
   - Do NOT paste the source code of your scripts into the report; the code is submitted as script files.
   - Screenshots must be readable full captures (not cropped and not photos taken by phone).
2. **Scripts:** all scripts must be error-free and include inline comments explaining the logic.
3. **Submission:** submit the report and all scripts on Blackboard in one zipped file named `StudentName1_StudentName2_StudentName3_StudentName4.zip`, no later than **14 October 2026**.

Penalties and Rules:

- Copying and/or plagiarism (-100%), which includes: inappropriate interaction with any other student, outside agency, website, or software that generates assessment responses.
- Shell scripts must be error free (Errors) (-25%).
- In case of late submission, (-10%) for each day of delay (Max 3 days delay).

- A group of three to four students can work on the project.
- Team members are required to meet regularly for discussion and workload distribution.
- It is the right of the instructor to use any way of testing the student in the discussion and demo session, and according to that in some cases a (100%) graded project may be brought down to a (-100%) graded project.
- Demo Session: randomly assigned tasks must be executed live, and students must explain every line of code.
- Discussion & Demo 50% + Student Work 50%.